

KHAI BÁO

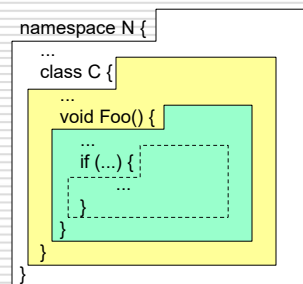
1

1

PHẠM VI KHÁI BÁO

□ Phân loại phạm vi khai báo

- **namespace**: khai báo class, interface, struct, enum, delegate
- **class, interface, struct**: khai báo trường dữ liệu, phương thức, ...
- **enumeration**: khai báo các hằng thuộc enum
- **khởi lệnh**: khai báo các biến cục bộ



2

2

CÁC NGUYÊN TẮC KHAI BÁO

□ Nguyên tắc về phạm vi khai báo

- Không có hai tên trùng nhau trong cùng một phạm vi và cùng một mức
- Có thể khai báo trùng tên khi ở các mức khác nhau

□ Nguyên tắc về tầm hoạt động

- Tên chỉ có tác dụng trong phạm vi khai báo của nó.
- Tầm hoạt động còn bị ảnh hưởng bởi các modifier như: *public*, *private*, *protected* và *internal*.

3

3

VÍ DỤ

- Các namespace ở file khác nhau tạo nên phạm vi khai báo khác nhau
- Namespace lồng tạo nên phạm vi khai báo của riêng nó

File: XXX.cs

```
namespace A {
    ... classes ...
    ... interfaces ...
    ... structs ...
    ... enumerations ...
    ... delegates ...
}
```

File: YYY.cs

```
namespace A {
    ...
}
namespace C {...}
```

4

4

SỬ DỤNG NAMESPACE

Color.cs

```
namespace Util {
    public enum Color {...}
}
```

Figures.cs

```
namespace Util.Figures {
    public class Rect {...}
    public class Circle {...}
}
```

Triangle.cs

```
namespace Util.Figures {
    public class Triangle {...}
}
```

using Util.Figures;

```
class Test {
    Rect r;           // không cần sử dụng tên đầy đủ (vì đã sử dụng Util.Figures)
    Triangle t;
    Util.Color c; // tên đầy đủ
}
```

- ❑ Đối với các namespace ngoài
 - Phải bổ sung thêm, dùng từ khoá using (e.g. *using Util;*)
 - sử dụng tên đầy đủ (e.g. *Util.Color*)
- ❑ Hầu hết các chương trình phải sử dụng namespace System => *using System;*

5

5

CLASS, INTERFACE, STRUCT

```
class C
{ // có thể áp dụng cho cả struct
    ... Trường dữ liệu, hằng ...
    ... Phương thức ...
    ... Hàm tạo, hàm huỷ ...
    ... Thuộc tính ...
    ... Indexer ...
    ... Sự kiện ...
    ... Chống toán tử ...
    ... Các kiểu lồng (class, interfaces struct, enum, delegate) ...
}
```

```
interface IX
{
    ... Phương thức ...
    ... Thuộc tính ...
    ... Indexer ...
    ... Sự kiện ...
}
```

- ❑ Phạm vi khai báo của subclass không phụ thuộc vào phạm vi khai báo của base class => có thể khai báo tên trùng nhau trong subclass và base class

6

6

KHỐI LỆNH

Các loại khối lệnh

```
void foo (int x) { // khối lệnh của phương thức B1
    ... local variables ...
    if (...) { // khối lệnh lồng B2
        ... local variables ...
    }
    for (int i = 0; ... ) { // khối lệnh lồng B3
        ... local variables ...
    }
}
```



- ☐ Phạm vi khai báo của khối lệnh bao gồm phạm vi khai báo của các khối lệnh lồng trong nó.
- ☐ Các tham số hình thức phụ thuộc vào phạm vi khai báo của khối lệnh của phương thức.
- ☐ Các biến được sử dụng trong vòng lặp phụ thuộc vào khối lệnh của vòng lặp
- ☐ Phải khai báo biến cục bộ trước khi sử dụng chúng

7

7

KHAİ BÁO BIẾN CỤC BỘ

```
void foo(int a) {
    int b;
    if (...) {
        int b; // lỗi: b đã được khai báo ở khối lệnh bên ngoài
        int c;
        int d;
        ...
    } else {
        int a; // lỗi: a là tham số
        int d; // đúng: không xung đột với d ở lệnh if
    }
    for (int i = 0; ... ) { ... }
    for (int i = 0; ... ) { ... } // đúng: không xung đột với i ở vòng lặp trước
    int c;
}
```

8

8